Chapter 14 Complete Solutions - Matrix Decompositions

This solution set provides worked Python solutions for every problem in the Chapter 14 matrix decomposition homework assignment.

The main concepts are LU decomposition, QR decomposition, Cholesky decomposition, reconstruction checks, matrix ranks, determinants, and why decompositions are useful in numerical linear algebra.

The code is written in a teaching style with comments and printed explanations so Lexie can connect the decomposition outputs to their interpretation.

Complete Python Solution Code

```
# Chapter 14 Complete Solutions - Matrix Decompositions
# Student: Lexie
# Purpose: Worked solutions for LU, QR, and Cholesky matrix decomposition,
#          plus a practical sensor matrix factorization problem.

import numpy as np
from scipy.linalg import lu
from numpy.linalg import qr, cholesky, matrix_rank, det, norm


def section(title):
    print("\n" + "=" * 80)
    print(title)
    print("=" * 80)


def problem1_lu_decomposition():
    section("Problem 1 - LU Decomposition")

    A = np.array([
        [2, 1, 1],
        [4, -6, 0],
        [-2, 7, 2]
    ], dtype=float)

    P, L, U = lu(A)
    A_reconstructed = P @ L @ U

    print("A =")
    print(A)
    print("\nP =")
    print(P)
    print("\nL =")
    print(L)
    print("\nU =")
    print(U)
    print("\nP @ L @ U =")
    print(A_reconstructed)
    print("\nReconstruction error =", norm(A - A_reconstructed))

    print("\nExplanation:")
    print("LU decomposition factors A into triangular pieces. L is lower triangular,")
    print("U is upper triangular, and P records any row swaps used for numerical stability.")


def problem2_qr_decomposition():
    section("Problem 2 - QR Decomposition")

    A = np.array([
        [1, 2],
        [3, 4],
        [5, 6]
    ], dtype=float)

    Q, R = qr(A, mode='reduced')
    A_reconstructed = Q @ R

    print("A =")

    print(A)
    print("\nQ =")
    print(Q)
```

```
    print("\nR =")
    print(R)
    print("\nQ @ R =")
    print(A_reconstructed)
    print("\nReconstruction error =", norm(A - A_reconstructed))

    print("\nExplanation:")
    print("QR decomposition writes A as Q @ R. Q has orthonormal columns and R is")
    print("upper triangular. This is especially useful in least-squares problems.")


def problem3_cholesky_decomposition():
    section("Problem 3 - Cholesky Decomposition")

    A = np.array([
        [4, 2, 2],
        [2, 5, 1],
        [2, 1, 3]
    ], dtype=float)

    L = cholesky(A)
    A_reconstructed = L @ L.T

    print("A =")
    print(A)
    print("\nL =")
    print(L)
    print("\nL @ L.T =")
    print(A_reconstructed)
    print("\nReconstruction error =", norm(A - A_reconstructed))

    print("\nExplanation:")
    print("Cholesky decomposition writes A as L @ L.T. It requires A to be symmetric")
    print("and positive definite. Symmetry is required because L @ L.T is always symmetric.")


def problem4_compare_shapes():
    section("Problem 4 - Compare Shapes")

    A_lu = np.array([
        [2, 1, 1],
        [4, -6, 0],
        [-2, 7, 2]
    ], dtype=float)
    P, L_lu, U = lu(A_lu)

    A_qr = np.array([
        [1, 2],
        [3, 4],
        [5, 6]
    ], dtype=float)
    Q, R = qr(A_qr, mode='reduced')

    A_chol = np.array([
        [4, 2, 2],

        [2, 5, 1],
        [2, 1, 3]
    ], dtype=float)
    L_chol = cholesky(A_chol)

    print("LU input A_lu.shape =", A_lu.shape)
    print("P.shape =", P.shape, "L.shape =", L_lu.shape, "U.shape =", U.shape)

    print("\nQR input A_qr.shape =", A_qr.shape)
    print("Q.shape =", Q.shape, "R.shape =", R.shape)

    print("\nCholesky input A_chol.shape =", A_chol.shape)
    print("L.shape =", L_chol.shape)

    print("\nExplanation:")
    print("LU is normally used for square matrices and produces square factors.")
    print("QR can be used for rectangular matrices. With reduced mode, Q has the")
    print("same number of rows as A but only as many columns as A has features.")
    print("Cholesky is for square symmetric positive definite matrices and returns")
    print("one triangular factor L.")


def part_b_sensor_correlation_compression():
    section("Part B - Practical Problem: Sensor Correlation Compression")

    A = np.array([
```

```
        [12, 15, 18],
        [14, 17, 22],
        [16, 20, 25]
    ], dtype=float)

    print("Sensor matrix A =")
    print(A)
    print("Columns: Radar, EO/IR, Acoustic")

    rank_A = matrix_rank(A)
    det_A = det(A)

    print("\nStep 1: rank(A) =", rank_A)
    print("Step 1: det(A) =", det_A)

    print("\nInterpretation:")
    print("The matrix has full rank if rank(A)=3. A determinant far from zero also")
    print("suggests the columns are not exact linear combinations of each other.")

    Q, R = qr(A, mode='reduced')

    print("\nStep 2: Q =")
    print(Q)
    print("\nStep 2: R =")
    print(R)

    print("\nInterpretation of Q and R:")
    print("Q contains orthonormal directions extracted from the original sensor columns.")
    print("R tells how to combine those orthonormal directions to reconstruct A.")

    A_reconstructed = Q @ R
    reconstruction_error = norm(A - A_reconstructed)

    print("\nStep 3: A_reconstructed = Q @ R =")
    print(A_reconstructed)
    print("Reconstruction error =", reconstruction_error)

    print("\nStep 4: Scaling explanation:")
    print("For very large matrices, directly computing inverses can be slow and")
    print("numerically unstable. Factorizations such as QR, LU, Cholesky, and SVD")
    print("break matrices into easier pieces and are often the preferred way to")
    print("solve systems or least-squares problems.")

    print("\nStep 5: Research paragraph:")
    print("Matrix decompositions are fundamental because they transform difficult")
    print("matrix problems into simpler structured problems. Triangular, orthogonal,")
    print("or diagonal factors are easier and more stable to work with than the")
    print("original matrix. This is why decompositions are used in solving linear")
    print("systems, least-squares regression, matrix inverses, determinants, PCA,")
    print("SVD, and many machine-learning algorithms.")


def main():
    problem1_lu_decomposition()
    problem2_qr_decomposition()
    problem3_cholesky_decomposition()
    problem4_compare_shapes()
    part_b_sensor_correlation_compression()


if __name__ == "__main__":
    main()
```

Expected Console Output

```
================================================================================
Problem 1 - LU Decomposition
================================================================================
A =
[[ 2.  1.  1.]
 [ 4. -6.  0.]
 [-2.  7.  2.]]

P =
[[0. 1. 0.]
 [1. 0. 0.]
 [0. 0. 1.]]

L =
[[ 1.   0.   0. ]
 [ 0.5  1.   0. ]
 [-0.5  1.   1. ]]

U =
[[ 4. -6.  0.]
 [ 0.  4.  1.]
 [ 0.  0.  1.]]

P @ L @ U =
[[ 2.  1.  1.]
 [ 4. -6.  0.]
 [-2.  7.  2.]]

Reconstruction error = 0.0

Explanation:
LU decomposition factors A into triangular pieces. L is lower triangular,
U is upper triangular, and P records any row swaps used for numerical stability.

================================================================================
Problem 2 - QR Decomposition
================================================================================
A =
[[1. 2.]
 [3. 4.]
 [5. 6.]]

Q =
[[-0.16903085  0.89708523]
 [-0.50709255  0.27602622]
 [-0.84515425 -0.34503278]]

R =
[[-5.91607978 -7.43735744]
 [ 0.          0.82807867]]

Q @ R =
[[1. 2.]
 [3. 4.]
 [5. 6.]]

Reconstruction error = 8.671119018262734e-16

Explanation:
QR decomposition writes A as Q @ R. Q has orthonormal columns and R is
upper triangular. This is especially useful in least-squares problems.

================================================================================
Problem 3 - Cholesky Decomposition
================================================================================
A =
[[4. 2. 2.]
 [2. 5. 1.]
 [2. 1. 3.]]

L =
[[2.         0.         0.        ]
 [1.         2.         0.        ]
 [1.         0.         1.41421356]]

L @ L.T =
[[4. 2. 2.]
 [2. 5. 1.]
 [2. 1. 3.]]
```

```
Reconstruction error = 4.440892098500626e-16

Explanation:
Cholesky decomposition writes A as L @ L.T. It requires A to be symmetric
and positive definite. Symmetry is required because L @ L.T is always symmetric.

================================================================================
Problem 4 - Compare Shapes
================================================================================
LU input A_lu.shape = (3, 3)
P.shape = (3, 3) L.shape = (3, 3) U.shape = (3, 3)

QR input A_qr.shape = (3, 2)
Q.shape = (3, 2) R.shape = (2, 2)

Cholesky input A_chol.shape = (3, 3)
L.shape = (3, 3)

Explanation:
LU is normally used for square matrices and produces square factors.
QR can be used for rectangular matrices. With reduced mode, Q has the
same number of rows as A but only as many columns as A has features.
Cholesky is for square symmetric positive definite matrices and returns
one triangular factor L.

================================================================================
Part B - Practical Problem: Sensor Correlation Compression
================================================================================
Sensor matrix A =
[[12. 15. 18.]
 [14. 17. 22.]
 [16. 20. 25.]]
Columns: Radar, EO/IR, Acoustic

Step 1: rank(A) = 3

Step 1: det(A) = -6.0

Interpretation:
The matrix has full rank if rank(A)=3. A determinant far from zero also
suggests the columns are not exact linear combinations of each other.

Step 2: Q =
[[-4.91539152e-01  3.44077407e-01 -8.00000000e-01]
 [-5.73462344e-01 -8.19231921e-01  1.30108500e-14]
 [-6.55385536e-01  4.58769875e-01  6.00000000e-01]]

Step 2: R =
[[-24.41311123 -30.22965787 -37.84851473]
 [  0.           0.40961596  -0.36046205]
 [  0.           0.           0.6       ]]

Interpretation of Q and R:
Q contains orthonormal directions extracted from the original sensor columns.
R tells how to combine those orthonormal directions to reconstruct A.

Step 3: A_reconstructed = Q @ R =
[[12. 15. 18.]
 [14. 17. 22.]
 [16. 20. 25.]]
Reconstruction error = 1.109334490649587e-14

Step 4: Scaling explanation:
For very large matrices, directly computing inverses can be slow and
numerically unstable. Factorizations such as QR, LU, Cholesky, and SVD
break matrices into easier pieces and are often the preferred way to
solve systems or least-squares problems.

Step 5: Research paragraph:
Matrix decompositions are fundamental because they transform difficult
matrix problems into simpler structured problems. Triangular, orthogonal,
or diagonal factors are easier and more stable to work with than the
original matrix. This is why decompositions are used in solving linear
systems, least-squares regression, matrix inverses, determinants, PCA,
SVD, and many machine-learning algorithms.
```